# *STRIPE* PROJECT

# 2. OLAP Data System

## D3. OLAP System Schema Design

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D3 : OLAP System Schema Design / v08 --- 2026/11/05.*

---

### D3.1. Contexte

Le présent document présente le schéma de l'entrepôt analytique :

- organisation du schéma en étoiles,
- stratégie d'agrégation,
- techniques d'optimisation des requêtes.

> Convention de notation des cardinalités dans les diagrammes présentés : notation *crow's foot*,
>
> - `>` `||` = un et un seul, `o{` = zéro à plusieurs.

L'analyse préliminaire des processus métiers est donnée en annexe [D3-A]. Six processus métiers sont identifiés en se basant sur les catégories des données analytiques données dans le *business case* [R0] :

| # | Processus métier | Source de données analytiques | Événement déclencheur |
|---|---|---|---|
| P1 | Traitement d'un paiement | *Revenue Metrics* | Création d'une transaction |
| P2 | Évaluation du risque de fraude | *Customer Segmentation Data* | Calcul d'un score de fraude |
| P3 | Cycle de vie d'un abonnement | *Product Performance Metrics* | Arrivée à échéance d'un abonnement |
| P4 | Accès et modification de données | *Fraud Analysis Data* | Journalisation d'une consultation ou d'une modification |
| P5 | Traitement d'une demande de droit | *Compliance and Audit Logs* | Demande d'application d'un droit RGPD/CCPA |
| P6 | Traitement d'un incident de sécurité | *Compliance and Audit Logs* | Détection d'un incident de sécurité |

### D3.2. Schémas en étoile

Dans le schéma en étoiles (*star*), chaque *dimension* est directement reliée à la *table de faits*, sans hiérarchie de sous-dimensions normalisées.

**Justification**

L'architecture en flocons (*snowflake*) réduit la redondance au prix de jointures supplémentaires ; en lecture analytique, c'est l'inverse de l'objectif recherché (*TR1* [R0] : *"optimize performance across all systems"*). La dénormalisation assumée dans les dimensions coûte en stockage mais pas en temps de requête.

### D3.3. Les étoiles des processus métiers

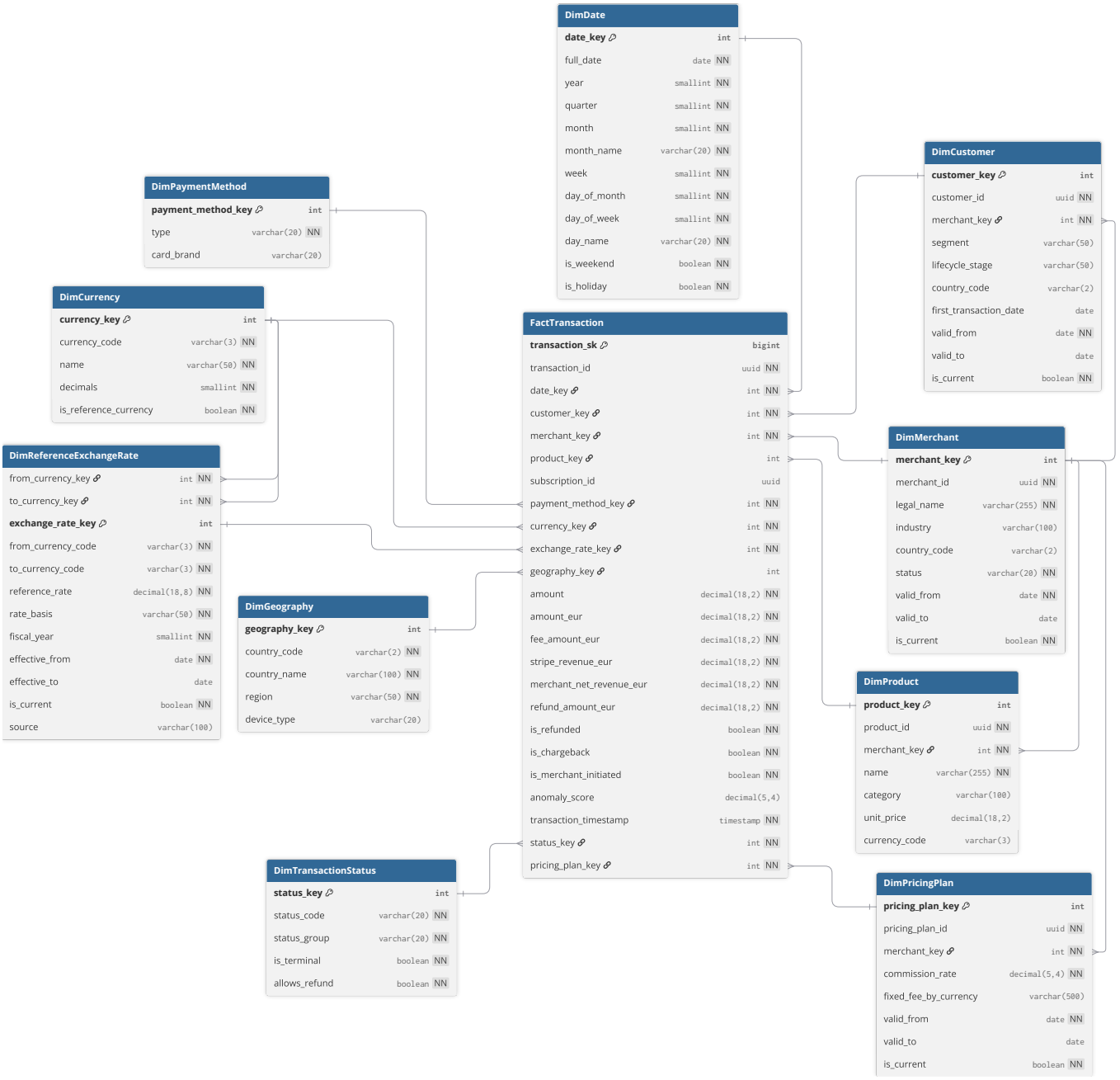
Les diagrammes ci-dessous présentent les schémas en étoile de chaque processus métier.

Les six étoiles sont regroupées en trois sous-ensembles fonctionnels :

- *Transactions*,
- *Abonnements & Fraudes*,
- *Sécurité & Conformité*.

> Rappel : chaque processus métier a son propre grain.

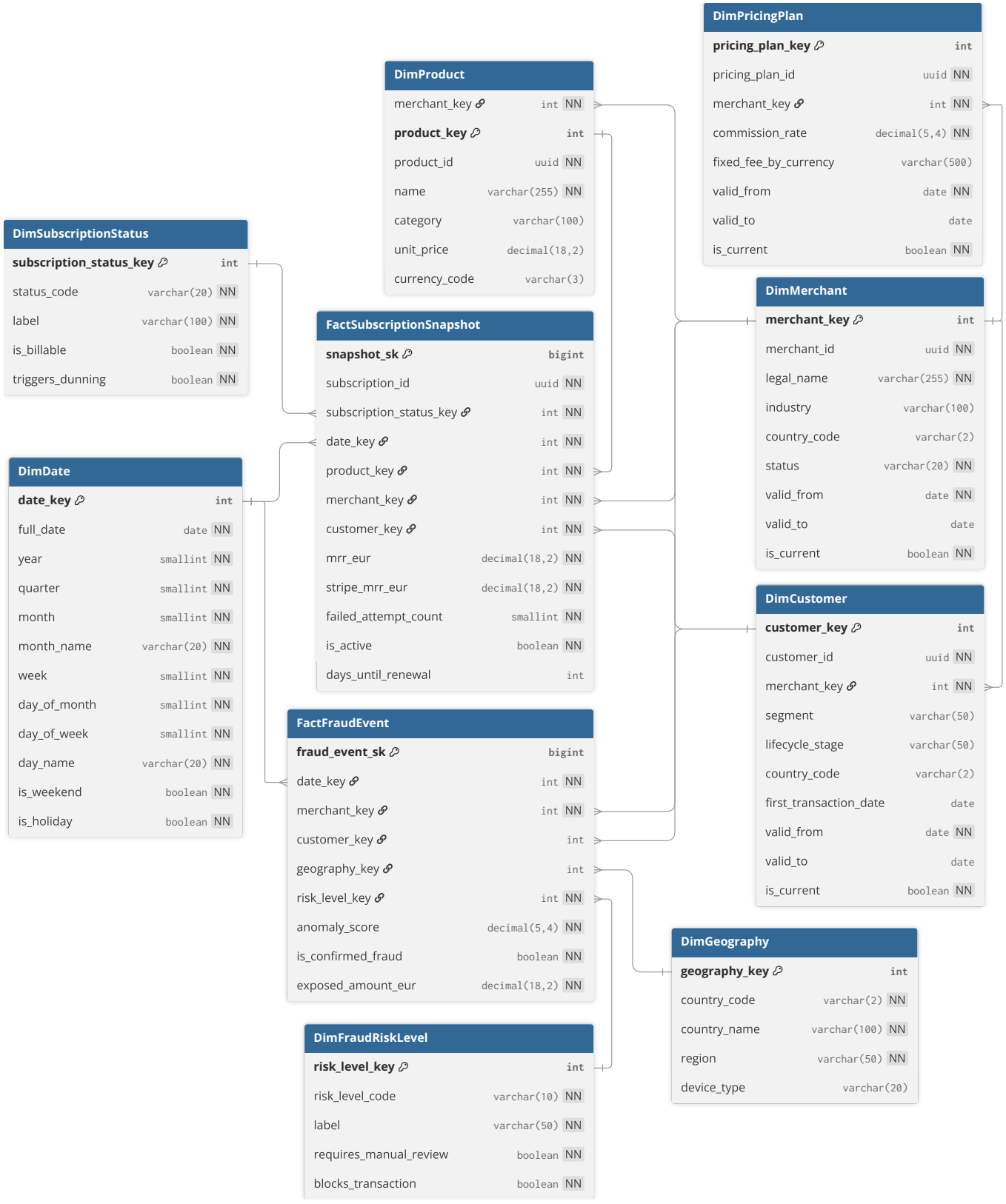
### D3.3.1. Revenus (P1/FactTransaction)




Grain : une transaction.

Les propriétés `merchant_net_revenue_eur` et `stripe_revenue_eur` cohabitent sur la même ligne. Les deux audiences du reporting (marchands & Stripe) lisent la même table, seule la colonne agrégée change (voir note dans le modèle physique[D3-B]).

### D3.3.2. Fraudes (P2/FactFraudEvent) & Abonnements (P3/FactSubscriptionSnapshot)

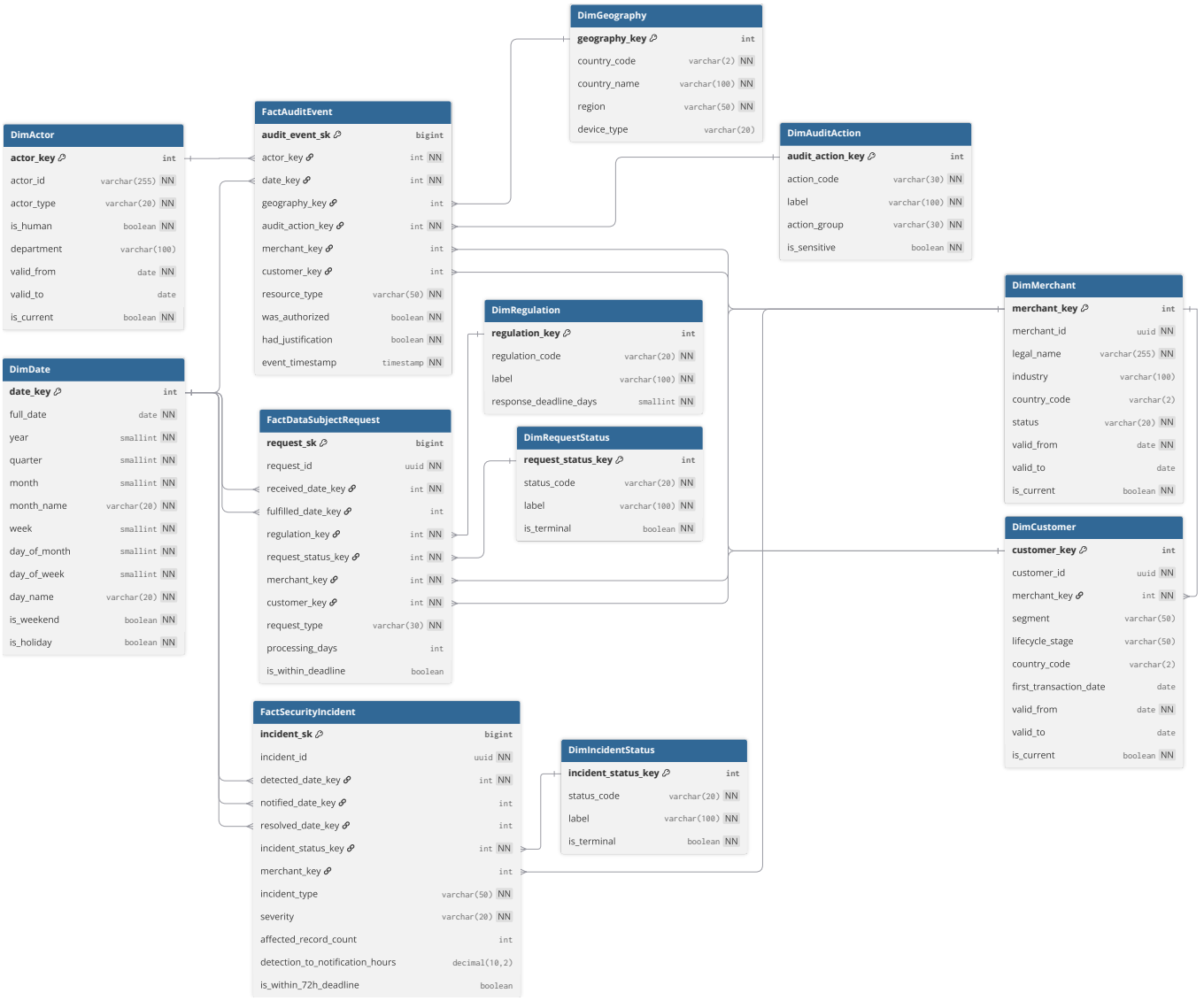



Deux grains différents dans la même étoile fonctionnelle :

- `FactFraudEvent` (un événement d'évaluation)
- `FactSubscriptionSnapshot` (un instantané d'abonnement capturé périodiquement).

Les tables ne partagent aucune ligne, seulement des dimensions conformées.

### D3.3.3. Conformité & Sécurité (P4/FactAuditEvent, P5/FactDataSubjectRequest, P6/FactSecurityIncident)




`FactDataSubjectRequest` et `FactSecurityIncident` référencent `DimDate` trois fois chacune (réception/traitement, détection/notification/résolution). C'est la signature d'un instantané cumulatif *Kimball* : plusieurs jalons temporels sur une même ligne, mise à jour au fur et à mesure du processus, au lieu de créer une nouvelle ligne par événement.

## D3.4. Stratégie d'agrégation

**Principe constant** : *le fait atomique reste au grain le plus fin, les agrégats sont des tables séparées, calculées.

**Dénormalisation différenciée entre fait atomique et agrégat** : le fait atomique référence toujours une dimension par clé ( `regulation_key` , par exemple) pour permettre n'importe quel angle d'analyse ; l'agrégat précalculé garde parfois l'attribut en clair (par exemple, `regulation` dans `AggDataSubjectRequestMonthly` ) pour pouvoir être interrogeable sans jointure (noter qu'un agrégat n'a pas vocation à être recombiné avec d'autres dimensions, il doit juste répondre vite à une question déjà posée).

Le tableau suivant expose les tables d'agrégation présentes dans le modèle physique [D3-B].

| Table d'agrégat | Grain | Alimentée depuis | Sert |
|---|---|---|---|
| `AggRevenueDailyMerchant` | Jour × marchand | `FactTransaction` | Tableau de bord marchand (quasi temps réel) |
| `AggRevenueMonthlyMerchantCountry` | Mois × marchand × pays | `FactTransaction` | Reporting géographique |
| `AggFraudDailyMerchant` | Jour × marchand | `FactFraudEvent` | Alerting fraude |
| `AggSubscriptionDailyMerchant` | Jour × marchand × produit | `FactSubscriptionSnapshot` | Tableau de bord abonnements |
| `AggComplianceDailyMerchant` | Jour × marchand | `FactAuditEvent` | Reporting conformité automatisé |
| `AggDataSubjectRequestMonthly` | Mois × marchand × réglementation × type | `FactDataSubjectRequest` | Rapport RGPD/CCPA |
| `AggSecurityIncidentMonthly` | Mois × marchand × sévérité | `FactSecurityIncident` | Rapport sécurité |

## D3.5. Techniques d'optimisation des requêtes

### D3.5.1. Clés de substitution entière

Déclarer les clés de substitution comme des entiers plutôt que des identifiants UUID.
Le format entier est nécessaires pour mettre en place un versionnement *SCD2* (un même `merchant_id` dans la base OLTP correspondra à plusieurs `merchant_key` dans la base OLAP, au fur et à mesure des modification de l'entité marchande).

### D3.5.2. `DimDate` précalculées

Un pré-calcul des informations d'horodatage dans `DimDate` évite les fonctions de date dans les clauses `WHERE` ( `EXTRACT` , `DATE_TRUNC` ...). Les fonctions de date empêchent l'*élagage de partitions* dans la plupart des moteurs orientés colonnes. `DimDate` permet aussi de faire des comparaisons période sur période par simple jointure sur `date_key` .

### D3.5.3. Partitionnement par `date_key`

Le partitionnement par `date_key` est systématique sur les tables de faits à fort volume ( `FactTransaction` , `FactAuditEvent` ). Il est mensuel ou quotidien selon le volume réel.
L'*élagage de partitions* élimine la majorité des lignes avant même le scan.

### D3.5.4. Stockage orienté colonnes

Systèmes courants : *Redshift*, *BigQuery*, *Snowflake* ou *ClickHouse* ; cohérent avec des requêtes qui agrègent peu de colonnes sur beaucoup de lignes (l'inverse du pattern *OLTP*).

### D3.5.5. Diffusion des dimensions

**Diffusion des dimensions (*broadcast join*).** Les dimensions restent petites en volume (quelques milliers à millions de lignes, jamais à l'échelle des faits) - elles peuvent être répliquées sur chaque nœud de calcul plutôt que redistribuées, évitant un *shuffle* coûteux lors de la jointure avec une table de faits partitionnée.

### D3.5.6. Taux de change de référence fixe

**Taux de change fixe ( `DimReferenceExchangeRate` ).** Au-delà de la comparabilité déjà justifiée (2.A.7), c'est aussi une optimisation : convertir au vol au taux du jour demanderait une jointure sur une dimension à cardinalité temporelle fine ; le taux figé par exercice réduit drastiquement le nombre de versions à joindre.

### D3.5.7. Sécurité RLS

**Sécurité au niveau des lignes (RLS), pas de filtrage applicatif.** Un marchand consultant son reporting ne doit voir que ses propres lignes ( `merchant_key` ) ; imposé par une politique RLS au niveau du moteur, jamais laissé à la charge de la requête applicative, pour qu'aucun bug côté client ne puisse exposer les données d'un autre marchand.

### D3.5.8. Index ciblés

**Index ciblés sur les clés de jointure et les colonnes de filtrage fréquent -** `date_key` et `(merchant_key, date_key)` systématiques sur les faits, complétés par les clés métier ( `transaction_id` , `subscription_id` ...) pour le dédoublonnage à l'ETL et le rapprochement avec l'OLTP.

## D3.6. Couverture des exigences

| Exigence [R0] | Couverture |
|---|---|
| **BR2** - analyse de revenu, segmentation, conformité | Six étoiles couvrant chacun des besoins (D3.3) |
| **TR1** - normalisation OLTP / performance OLAP | Dénormalisation assumée, justifiée section D3.2 |
| **TR3** - scalabilité, indexation, partitionnement | Partitionnement par `date_key` , index ciblés (D3.5) |

---

## Références

### Ressources

- [R0] Stripe Business Case
-

### Annexes

- [D3-A] OLAP Kimball Analysis
- [D3-B] OLAP Physical Data Model